Complete CI/CD Pipeline

Project Overview:

This project demonstrates the implementation of a complete CI/CD pipeline for deploying a Java application to a Kubernetes cluster using modern DevOps tools and best practices. The pipeline automates the processes of code building, testing, security scanning, artifact management, containerization, deployment, and monitoring.

The pipeline begins by compiling the application and executing unit tests using Maven to ensure that the source code builds successfully and passes all test cases.

After the build stage, SonarQube is used for static code analysis to evaluate code quality and detect potential bugs, vulnerabilities, and code smells.

Next, Trivy is used to perform security scanning of the source code and dependencies. This step helps identify vulnerabilities and detect any sensitive data present in the codebase. A detailed vulnerability report is generated during this stage.

Once the code passes quality and security checks, the application is packaged using Maven, generating a deployable artifact (JAR/WAR file). This artifact is then published to the Nexus Repository Manager, enabling centralized artifact storage and version control.

After artifact publication, a Docker image is built using the generated application artifact. The container image is then scanned using Trivy to detect vulnerabilities within the container environment.

Once the image is verified, it is pushed to Docker Hub, making it available for deployment.

Before deploying the application, the Kubernetes cluster is scanned using KubeAudit to ensure that the cluster configuration follows security best practices.

Following the security validation, the application is deployed to the Kubernetes cluster using automated deployment steps.

Upon successful deployment, the pipeline sends an email notification confirming the deployment status.

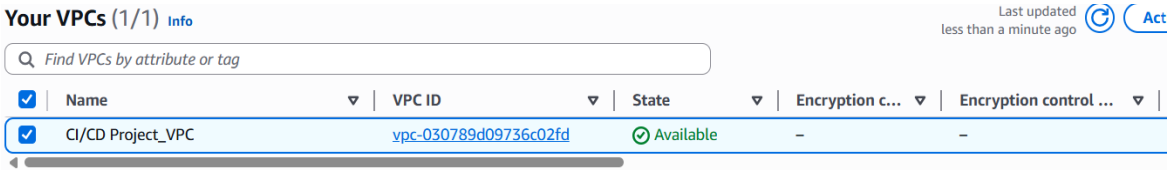
Finally, the deployed application and infrastructure are continuously monitored using Prometheus and Grafana:

- Application Availability Monitoring is implemented using the Blackbox Exporter, which verifies whether the website is accessible and measures response behaviour. The results are visualized in Grafana dashboards.
- System-Level Monitoring is implemented using the Node Exporter, which collects system metrics such as CPU usage and memory consumption. Jenkins server performance is monitored using these metrics.

This project demonstrates a production-style DevOps workflow, integrating CI/CD automation, security scanning, containerization, Kubernetes deployment, and observability.

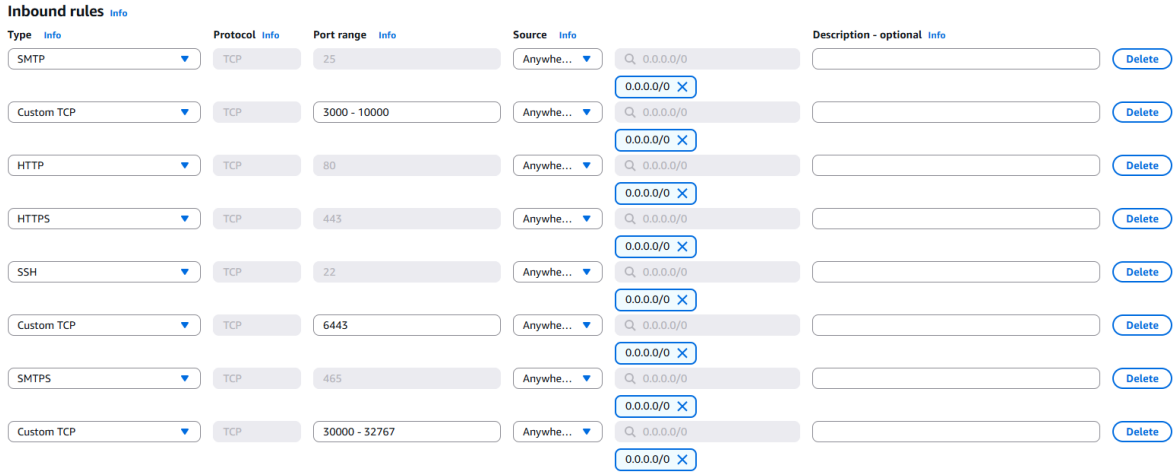
Phase 1: Setup the Infrastructure on AWS
Step 1: VPC

Go to VPC → There you will see a default VPC will be present, you can use this VPC itself. You can give a name to this VPC if you want.



Step 2: Create Security Group

Go to EC2 → Security groups → Create security group → Add all the below ports in Inbound rules



Here,
Port 30000 – 32767 → To expose applications running inside a Kubernetes cluster so they can be accessed from outside the cluster.
Port 465 → For email notifications.
Port 6443 → Allows tools like Jenkins, kubectl, Helm, or CI/CD pipelines to communicate with the Kubernetes cluster securely.
Port 22 → To access the EC2 Instances.
Port 3000 – 10000 → To expose web applications and tool dashboards (like Jenkins, SonarQube, Prometheus, and Grafana) so they can be accessed through a browser.

Step 3: Create EC2 Instances

Go to EC2 → Instances → Launch Instance → Select AMI as Ubuntu → Instance type as t2.medium → Select the security group that you just created → Select storage as 25 GB → Number of instances = 3 → Launch instance.
Once the instances are launched, name one as Master and other two as Slave1 and Slave2.

Step 4: Login in EC2 Instances

Login into all 3 EC2 Instances. You can use the MobaXterm tool to interact with your EC2 instances.

Step 5: Setup Kubernetes Cluster

On all 3 EC2 Instances i.e. Master, Slave1 and Slave2, create a file "1.sh" (you can give any name to this file). Write the below commands in this file.

```
>> sudo apt-get update
>> sudo apt install docker.io -y
>> sudo chmod 666 /var/run/docker.sock
>> sudo apt-get install -y apt-transport-https ca-certificates curl gpg
>> sudo mkdir -p -m 755 /etc/apt/keyrings
>> curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/Kubernetes-apt-keyring.gpg
```

>> echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /' | sudo tee /etc/apt/sources.list.d/Kubernetes.list
Save this file. Give the execute permissions to this file. >> chmod +x 1.sh
Execute the file. >> ./1.sh
Again run the command >> sudo apt update
Step 6: Install kubeadm, kubelet and kubectl
Run the below command on all 3 EC2 Instances, >> sudo apt install -y kubeadm=1.28.1-1.1 kubelet=1.28.1-1.1 kubectl=1.28.1-1.1 kubeadm → it is a tool used to create and initialize a Kubernetes cluster. kubelet → it is an agent that runs on each worker node in the Kubernetes cluster and manages the containers. kubectl → command-line tool used to interact with the Kubernetes cluster.
Run below command on the master EC2 Instance, >> sudo kubeadm init –pod-network-cidr=10.244.0.0/16 This command will start the control plane of the Kubernetes cluster and it will generate a join command for worker nodes. This join command you have to run on Slave1 and Slave2 EC2 Instances.
Now, run the kubeadm join command which is generated, on Slave1 and Slave2 machines >> kubeadm join 172.xx.xx.xxx:6443 –token ……. Once you run this command you will see a message "This node has joined the cluster"
Run below commands on Master >> mkdir -p $HOME/ .kube >> sudo cp -I /etc/Kubernetes/admin.conf $HOME/ .kube/config >> sudo chown $(id -u):$(id -g) $HOME/ .kube/config >> kubectl apply -f https://docs.projectcalico.org/v3.20/manifests/calico.yaml >> kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v0.49.0/deploy/static/provider/baremetal/deploy.yaml
Step 7: Scan the Kubernetes cluster for any issues using Kubeaudit
kubeaudit → This tool scans your Kubernetes configurations and tells you if there are security risks. Go to this github repository → github.com/Shopify/kubeaudit/releases Copy the link of linux_amd64 zip file from this repository. On the Master machine >> wget paste_the_link Unzip the tar file >> tar -xvzf file_name Move the kubeaudit to bin folder >> sudo mv kubeaudit /usr/local/bin Scan the cluster by running below command, >> kubeaudit all
Step 8: Launch 3 more EC2 Instances
1 for SonarQube, 1 for Nexus and another one for Jenkins. For SonarQube and Nexus,

Go to EC2 → Instances → Launch Instance → Select AMI as Ubuntu → Instance type as t2.medium → Select the security group that we created → Select storage as 25 GB → Number of instances = 2 → Launch instance.
Once the instances are launched, name one as SonarQube and other Nexus.

For Jenkins,
Go to EC2 → Instances → Launch Instance → Select AMI as Ubuntu → Instance type as t2.large → Select the security group that we created → Select storage as 30 GB → Number of instances = 1 → Launch instance.
Once the instance is launched, name it as JenkinsServer.

Step 9: Setup SonarQube and Nexus

Login into SonarQube and Nexus EC2 Instance.
>> sudo apt update

Install docker on both machines.
Go to browser → search install docker → click on official docker link → copy below commands

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Create a file on your EC2 Instance “new.sh” (you can give any name to this file)
Paste the commands in this file.
Save the file.
Give execute permissions to the file.
>> chmod +x new.sh
Execute the file
>> ./new.sh
>> sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
>> sudo chmod 666 /var/run/docker.sock

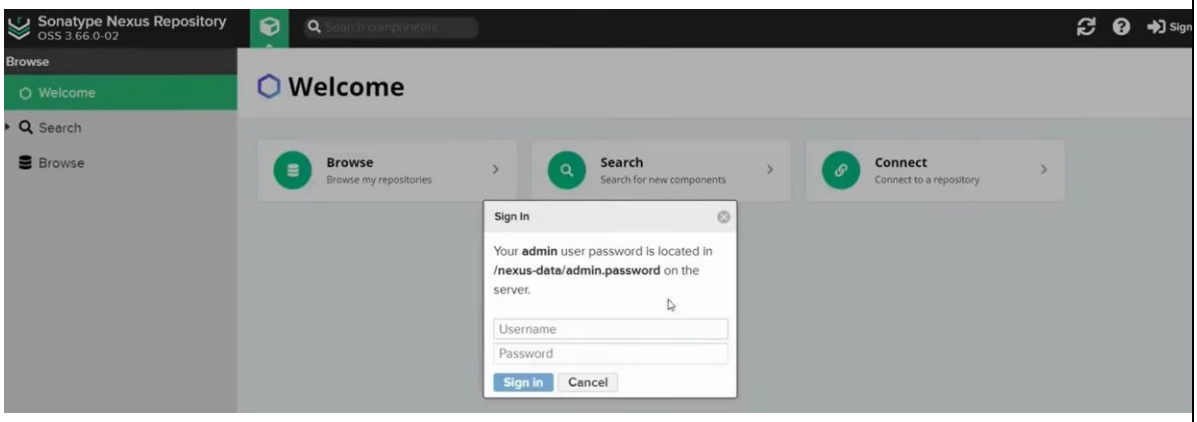
Now on SonarQube machine run below command,
>> docker run -d –name sonar -p 9000:9000 sonarqube:lts-community

Now on Nexus machine run below command,
>> docker run -d –name nexus -p 8081:8081 sonatype/nexus3

Now go to your browser → paste the public_IP_of_your_SonarQube_EC2_Instance:9000
The default username and password for SonarQube is admin.
Next it will ask you to change the password.

Again for Nexus, go to your browser → paste the public_IP_of_your_Nexus_EC2_Instance:8081
Click on Sign in, the default username is admin and password is present in /nexus-data/admin.password file. So go to your Nexus EC Instance and run below commands,
>> docker exec -it nexus /bin/bash

Now you are inside the container
>> ls
>> go to this path /nexus-data/admin.password
Copy the password



Step 10: Setup Jenkins

Now go to your Jenkins EC2 Instance,
>> sudo apt update

First install java
>> sudo apt install openjdk-17-jre-headless -y

Now, go to your browser → install Jenkins → Linux → Ubuntu → copy the commands
Create a file on your EC2 Instance "jenkins.sh". Paste the commands.

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
```

```
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins -y
```

Save the file.
Give execute permissions to the file.
>> chmod +x jenkins.sh
Execute the file.
>> ./jenkins.sh

Also install docker on this EC2 Instance

Go to browser → search install docker → click on official docker link → copy below commands

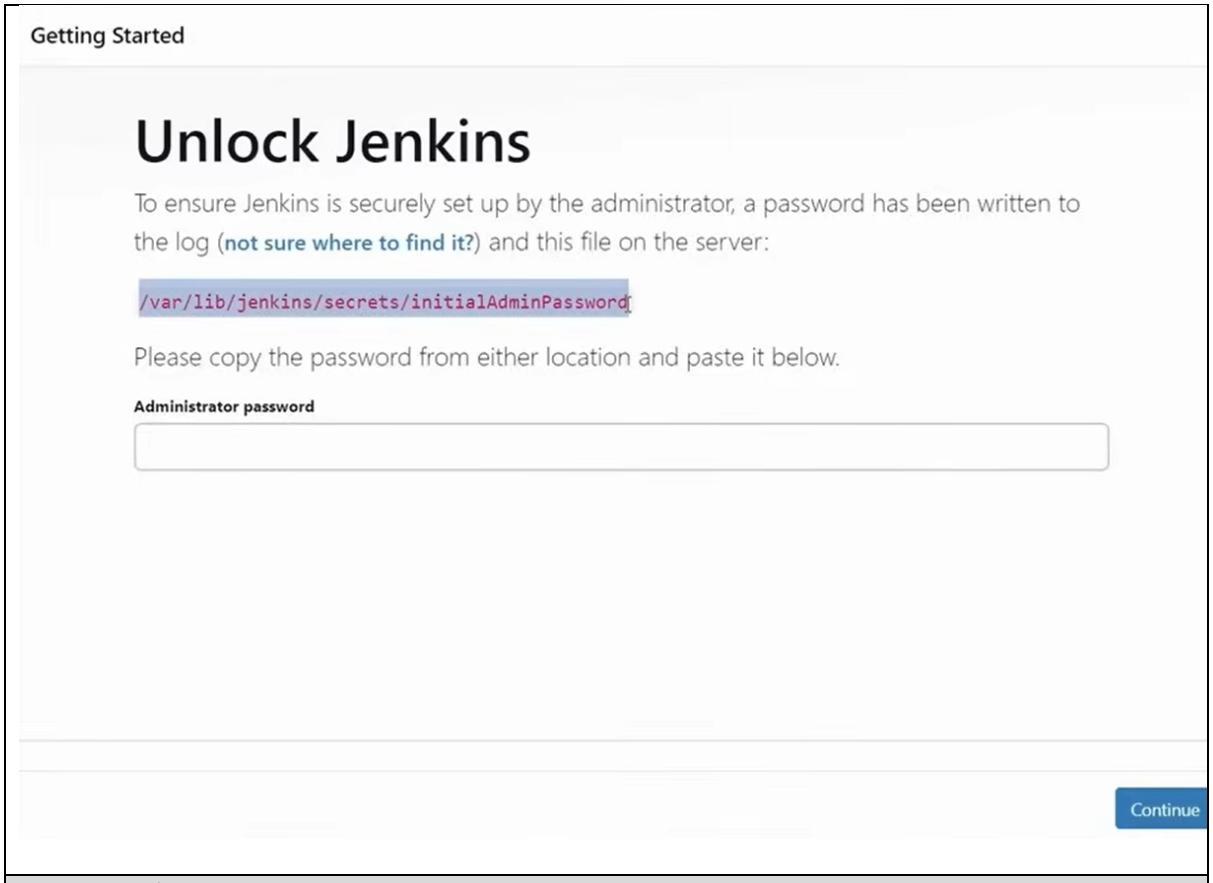
```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Create a file on your EC2 Instance “new.sh” (you can give any name to this file)
Paste the commands in this file.
Save the file.
Give execute permissions to the file.
>> chmod +x new.sh
Execute the file
>> ./new.sh
>> sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
>> sudo chmod 666 /var/run/docker.sock

Now go to your browser → paste the public_IP_of_your_Jenkins_EC2_Instance:8081
For Jenkins password, go to /var/lib/Jenkins/secrets/initialAdminPassword file and copy the password.



Phase 2: CI/CD Pipelines

Step 1: Install plugins in Jenkins

Once the Jenkins server is setup, go to → Manage Jenkins → Plugins → Search

- Eclipse Temurin Installer
- Config File Provider
- Pipeline Maven Integration
- Maven Integration
- SonarQube Scanner
- Docker
- Docker Pipeline
- Kubernetes Client API
- Kubernetes Credentials
- Kubernetes
- Kubernetes CLI

Select all these plugins and click on Install.

Step 2: Configure all the tools

Go to Manage Jenkins → Tools → JDK installations
Give name jdk17 → Install automatically → Click on Add Installer → select Install from adoptium.net → select jdk17

Now, Tools → SonarQube Scanner installations
Give name sonar-scanner

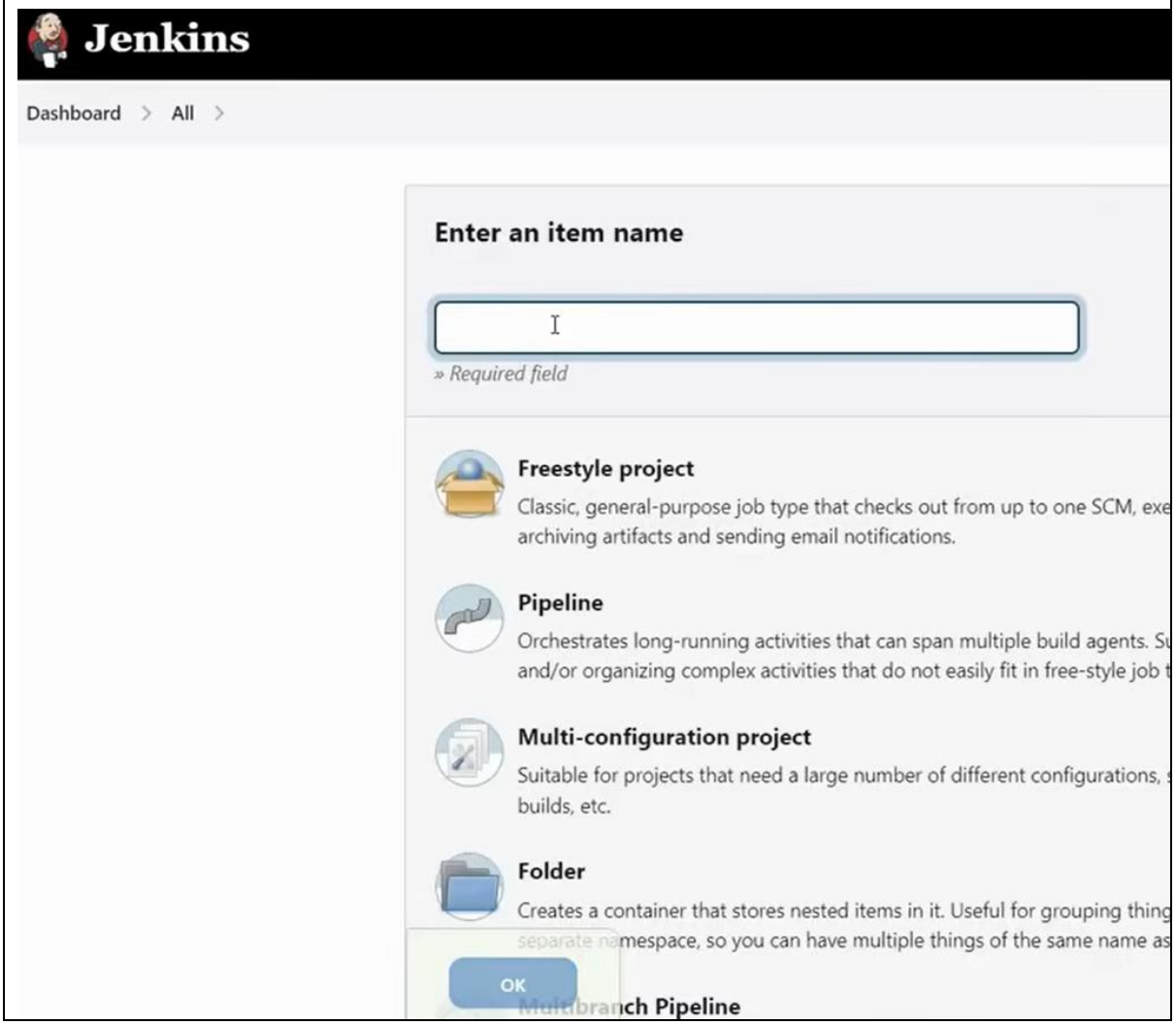
Now, Tools → Maven installations
Give name maven3

Now, Tools → Docker installations
Give name docker → Install automatically → Click on Add Installer → select Install from docker.com → select version

Click on Apply and Save.

Step 3: Create the pipeline.

Go to New Item → give name to the pipeline → select Pipeline → OK



Next select Discard old builds → Max # of builds to keep 2

Start writing the pipeline,

```
pipeline {
        agent any

        tools {
                jdk 'jdk17'
                maven 'maven3'
        }
        environment {
                SCANNER_HOME= tool 'sonar-scanner'
        }

        stages {
                stage('Git Checkout') {
                        steps{

                        }
                }
```

Click on Pipeline Syntax → Sample Step → select git → in repository URL, copy paste the URL of your git repository → in Branch, give the name of your git repository branch → in credential, select the git credential that you have created → click on Generate pipeline script
Copy the output and paste in your pipeline.

```
        stages {
                stage('Git Checkout') {
                        steps{
                            git branch: ‘main’, credentialsID: ‘git-cred’, url:
‘https://github.com/complete_url’
                        }
                }
```

[To create the credential, follow these steps: Go to Manage Jenkins → Credentials → click on global → Add credentials → give Username, Password, ID and Description. In Password, paste the git token → Create]

Next stage in pipeline is compilation of code,

```
                stage('Compile') {
                        steps{
                            sh “mvn compile”
                        }
                }
```

Compilation is done to find out if there are any syntax errors in the code.

Next stage in pipeline is testing the code,

```
                stage('Test') {
                        steps{
                            sh “mvn test”
                        }
                }
```

Next stage in the pipeline is to check if there is any sensitive data present. For this, the tool that is used is Trivy.
The default plugin is not available for Trivy, so we have to install Trivy on our Jenkins EC2 Instance.
Go to Jenkins EC2 Instance,
Create a file on your EC2 Instance “tri.sh” (you can give any name to this file)
Paste the commands in this file.
Save the file.
Give execute permissions to the file.
>> chmod +x tri.sh
Execute the file
>> ./tri.sh

Next go to your pipeline and add below stage,

```
stage('File system scan') {
        steps{
            sh “trivy fs –format table -o trivy-fs-report.html .”
        }
}
```

Next stage in pipeline is to scan the code to find out if there are any vulnerabilities in the dependencies that we are using,

```
stage('SonarQube Analysis') {
        steps{

        }
}
```

The dependencies that are used by the code are stored in pom.xml file.

Setup the credentials for SonarQube,
Go to Manage Jenkins → Credentials → global → Add Credentials → select Secret text → paste the SonarQube token in Secret → give ID and Description → Create

Configure SonarQube server inside Jenkins,
Go to Manage Jenkins → System → SonarQube servers → Add SonarQube → give name → server URL will be ‘public_IP_of_SonarQube_EC2_Instance:9000’ → In server authentication token, select the token which you have created → Apply

Click on Pipeline Syntax → Sample Step → select ‘withSonarQubeEnv:’ → in server authentication token, select the credential that you have created → click on Generate pipeline script
Copy the output and paste in your pipeline.

```
stage('SonarQube Analysis') {
        steps{
            withSonarQubeEnv(‘sonar’) {
                sh ‘’’ $SCANNER_HOME/bin/sonar-scanner
-Dsonar.projectName=your_project_name -Dsonar.projectKey=same_as_project_name
-Dsonar.java.binaries=. ‘’’
            }
        }
}
```

Next stage is Quality gate,

Go to SonarQube server → Administration → Configuration → Webhooks → Create → give name → in URL, give public_IP_of_Jenkins_EC_Instance:8080/sonarqube-webhook/

Next go to your pipeline and add below stage,

```
stage('Quality Gate') {
        steps{
            script{
            waitForQualityGate abortPipeline: false, credentialsId: 'sonar-token'
            }
        }
}
```

Quality gate of SonarQube is like conditions. If these conditions pass then we can say that our code is of good quality.

Next stage in pipeline is to build the code,

```
stage('Build') {
        steps{
            sh "mvn package"
        }
}
```

Next stage in pipeline is to publish the artifact to Nexus repository,

```
stage('Publish to Nexus') {
        steps{

        }
}
```

To publish our artifact to Nexus repository, we have to add the repository URL to pom.xml file
Go to Nexus server → copy URL of maven-releases → go to pom.xml, <distributionManagement>, there paste the URL
Go to Nexus server → copy URL of maven-snapshots → go to pom.xml, <distributionManagement>, there paste the URL

Go to Manage Jenkins → Managed files → Add a new Config → select Global Maven settings.xml → give ID → Next → in Content, find servers and write

```
<server>
<id>maven-releases</id>
<username>admin</username>
<password>enter the password that you have set for the Nexus server</password>
</server>

<server>
<id>maven-snapshots</id>
<username>admin</username>
<password>enter the password that you have set for the Nexus server</password>
</server>
```

Click on Submit.

Go to pipeline,

Click on Pipeline Syntax → Sample Step → select 'withMaven:' → select maven3 → select jdk17 → in Global Maven settings config, select MyGlobalSettings → click on Generate pipeline script
Copy the output and paste in your pipeline.

```
stage('Publish to Nexus') {
        steps{
            withMaven(globalMavenSettingsConfig: 'global-settings', jdk: 'jdk17',
maven: 'maven3', mavenSettingsConfig:'', traceability: true) {
                sh "mvn deploy"
            }
        }
}
```

Next stage in pipeline is to build the docker image,

```
stage('Build and Tag Docker Image') {
        steps{

        }
}
```

Setup the credentials for Docker,
Go to Manage Jenkins → Credentials → global → Add Credentials → select Username and Password → give Username, Password, ID and Description → Create

Click on Pipeline Syntax → Sample Step → select 'withDockerRegistry:' → in Docker registry URL, give the URL of your private Docker registry → in Registry credentials, select the credentials that you have created → click on Generate pipeline script
Copy the output and paste in your pipeline.

```
stage('Build and Tag Docker Image') {
        steps{
            script{
            withDockerRegistry(credentialsId: 'docker-cred', toolName: 'docker') {
                sh "docker build -t dockerhub_username/image_name:tag ."
            }
        }
}
```

Next stage in pipeline is to scan the image,

```
stage('Docker Image scan') {
        steps{
            sh "trivy image –format table -o trivy-image-report.html
your_image_name"
        }
}
```

Next stage in pipeline is to push the image to the docker hub repository,

```
stage('Push Docker Image') {
        steps{
            sh "docker push dockerhub_username/image_name:tag"
        }
}
```

Now the Continuous Integration part is complete. Next comes the part of Continuous Deployment. Deploy the application on Kubernetes cluster.

RBAC → Role Based Access Control
First, we need to create Service Account.

Go to Master EC2 Instance,
Create a file, vi svc.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
    name: Jenkins
    namespace: webapps
```

Save the file.

Create the namespace,
>> kubectl create ns webapps
>> kubectl apply -f svc.yaml

Next, we need to create a Role.
Create a file, vi role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
    name: app-role
    namespace: webapps
rules:
 -   apiGroups:
        “”
        apps
        autoscaling
        batch
        extensions
        policy
        rbac,authorization.k8s.io
    resources:
        pods
        componentstatuses
        congifmaps
        daemonsets
        deployments
        events
        endpoints
        ingress
        jobs
        replicasets
        nodes
        serviceaccounts
        persistentvolumes
    verbs: [“get”, “list”, “watch”, “create”, “update”, “patch”, “delete”]
```

Save the file.

>> kubectl apply -f role.yaml

Now, we have to assign the role to the service account.
Create a file, vi bind.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Rolebinding
metadata:
    name: app-rolebinding
    namespace: webapps
roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: Role
    name: app-role
subjects:
 -   namespace: webapps
     kind: ServiceAccount
     name: jenkins
```

Save the file.

>> kubectl apply -f bind.yaml

Now, we need to create a credential of Kubernetes for Jenkins
Create a file, vi sec.yaml

```
apiVersion: v1
kind: Secret
type: Kubernetes.io/service-account-token
metadata:
        name: mysecretname
        annotations:
          Kubernetes.io/service-account.name: Jenkins
```

Save the file

>> kubectl apply -f sec.yaml -n webapps

>> kubectl describe secret mysecretname -n namespace

Copy the token.

Go to Manage Jenkins → Credentials → global → Add Credentials → select Secret text → paste the token → give ID and Description → Create

>> cd ~/.kube
>> ls
>> cat config

Copy the server endpoint.

Click on Pipeline Syntax → Sample Step → select 'withKubeConfig:' → in Credentials, select the credentials that you have created → in Kubernetes server endpoint, paste the endpoint → in Cluster name, copy-paste the Kubernetes cluster name → in Namespace, give the name webapps → click on Generate pipeline script
Copy the output and paste in your pipeline.

Now, in the pipeline add below stage,

```
stage('Deploy to Kubernetes') {
    steps{
        withKubeConfig(caCertificate: '', clusterName: 'kubernetes',
contextName: '', credentialsId: 'k8-cred', namespace: 'webapps', restrictKubeConfigAccess: false,
serverUrl: 'https://172.31.5.168:6443') {
            sh "kubectl apply -f deployment-service.yaml"
        }
    }
}
```

Now, we also need to install kubectl on Jenkins,
Go to Jenkins server EC2 Instance,
Create a file, vi k.sh
curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin
kubectl version –short --client
Save the file.
Give execute permissions to the file.
>> chmod +x k.sh
Execute the file
>> ./k.sh

Next stage in pipeline is to check whether the application has been deployed successfully.

```
stage('Verify the deployment') {
    steps{
        withKubeConfig(caCertificate: '', clusterName: 'kubernetes',
contextName: '', credentialsId: 'k8-cred', namespace: 'webapps', restrictKubeConfigAccess: false,
serverUrl: 'https://172.31.5.168:6443') {
            sh "kubectl get pods"
            sh "kubectl get svc"
        }
    }
}
```

Next, we have to setup the email notifications,
Go to your google account → security → two-step verification → App passwords → give App name → Create → Copy the password

Go to Manage Jenkins → System → Extended Email notification → in SMTP server, give smtp.gmail.com → in SMTP port, give 465 → click on Advanced, select Use SSL → Credentials → Add → in Username, give the gmail email Id → paste the password → give ID and Description → select the credential that you just created

Scroll down to Email notification → in SMTP server, give smtp.gmail.com → click on Advanced, select Use SSL → select Use SMTP Authentication → in Username, give the gmail email Id → paste the password → in SMTP port, give 465 → Save

Phase 3: CI/CD Pipelines

Step 1: Launch an EC2 Instance

Go to EC2 → Instances → Launch Instance → Select AMI as Ubuntu → Instance type as t2.medium → Select the security group that we created → Select storage as 25 GB → Number of instances = 1 → Launch instance.
Once the instance is launched, name it as Monitor.

Login into the EC Instance.

Step 2: Install Prometheus and Grafana

Run below commands,
>> sudo apt update

Go to browser → install Prometheus → go to the official website → copy the link of the tar file

>> wget paste the link
>> tar -xvf filename
>> rm -rf filename.tar
>> ./prometheus &

Copy the IP address of the EC2 Instance and paste it in the brower:9090
So, Prometheus is up and running

Now install Grafana
Go to browser → install Prometheus → go to the official website → copy the commands

>> sudo apt-get install -y adduser libfontconfig1 musl
>> wget https://dl.grafana.com/enterprise/release/grafana-enterprise_10.4.0_amd64.deb
>> sudo dpks -i grafana-enterprise_10.4.0_amd64.deb
>> sudo /bin/systemctl start grafana-server

Copy the IP address of the EC2 Instance and paste it in the brower:3000
So, Grafana is up and running.
Default password would be admin.

Step 3: Install blackbox exporter

On the Prometheus website, copy the link of blackbox exporter tar file

>> wget paste the link
>> tar -xvf filename
>> rm -rf filename.tar
>> ./blackbox_exporter &

Copy the IP address of the EC2 Instance and paste it in the brower:9115
So, Blackbox Exporter is up and running.

Edit the prometheus.yaml file. Add blackbox export part in this yaml file. Mention the URL of our application that we will be monitoring.

Restart Prometheus
>> pgrep prometheus
>> kill paste_the_id
>> ./prometheus &

Step 4: Add Prometheus as data source inside Grafana

Go to Grafana → Connections → Data sources → Prometheus → give the IP address of the Prometheus 'IP address of the EC2 Instance:9090' → click on Save and Test
Go to browser → blackbox Grafana dashboard → click on the official link → click on copy ID to clipboard

Now, go to Grafana → Import dashboard → paste the ID → click on Load → select Prometheus → Import

Now you will be able to see the graphs.

Step 5: Monitor Jenkins

Go to Manage Jenkins → Plugins → search Prometheus → Install

On the Prometheus website, copy the link of node exporter tar file

Go to Jenkins EC2 Instance,
>> wget paste the link
>> tar -xvf filename
>> rm -rf filename.tar
>> ./node_exporter &

Copy the IP address of EC2 Instance → paste in the browser:9100
Node exporter is running.

Go to Jenkins → Manage Jenkins → System → Prometheus → Review the selections → Apply

Go to Monitor EC2 Instance
Edit the prometheus.yaml file. Add node export part in this yaml file. Mention the URL of Jenkins server
Restart Prometheus
>> pgrep prometheus
>> kill paste_the_id
>> ./prometheus &

Go to browser → node exporter Grafana dashboard → click on the official link → click on copy ID to clipboard

Now, go to Grafana → Import dashboard → paste the ID → click on Load → select Prometheus → Import

Now you will be able to see the graphs.